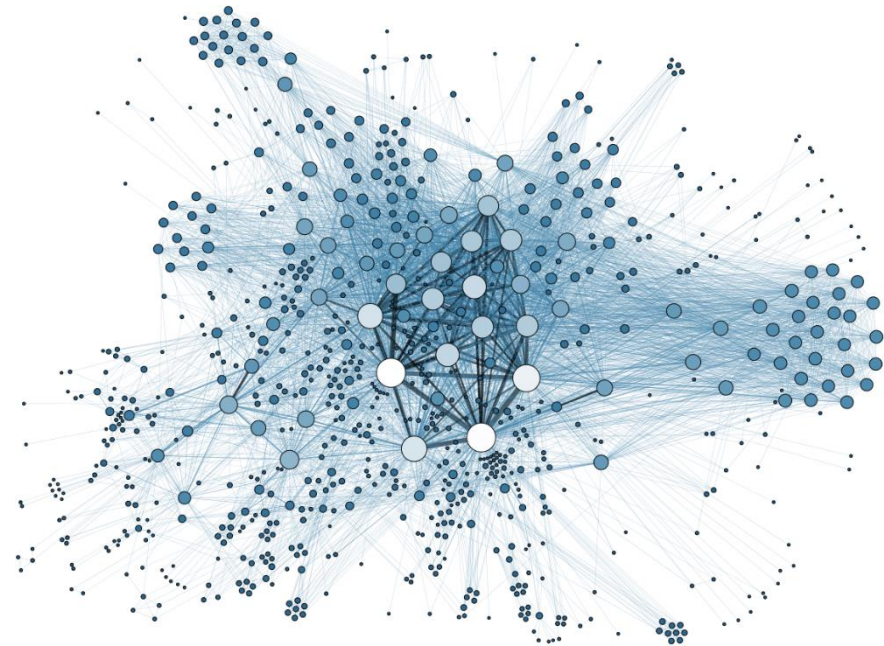




Community Analysis

SE4095-Social Network: Theory and Practice



Social Community



[real-world] community

A group of individuals with common *economic*, *social*, or *political* interests or characteristics, often living in *relative proximity*.

Why analyze communities?

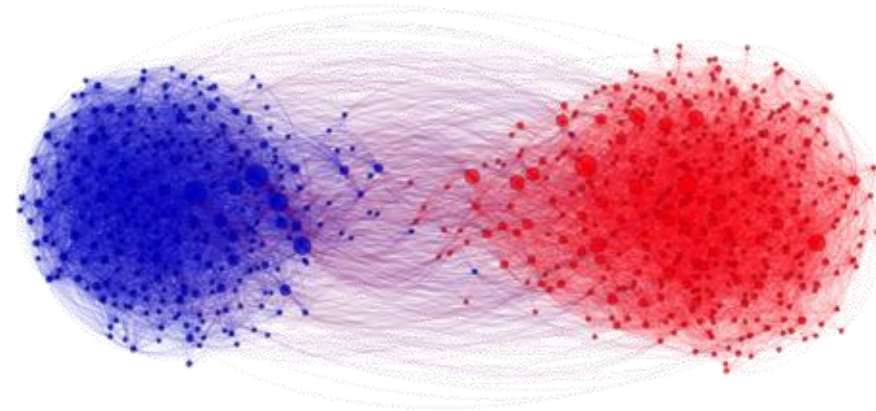


Analyzing communities helps better understand users

- Users form groups based on their interests

Groups provide a clear global view of user interactions

- E.g., find polarization



Some behaviors are only observable in a group setting and not on an individual level

- Some republican can **agree** with some democrats, but their parties can **disagree**

Social Media Communities

- **Formation:**
 - When like-minded users on social media form a link and start interacting with each other
- **More Formal Formation:**
 1. A set of at least two nodes sharing some interest, and
 2. Interactions with respect to that interest.
- Social Media Communities
 - **Explicit** (*emic*): formed by user subscriptions
 - **Implicit** (*etic*): implicitly formed by social interactions
 - **Example:** individuals calling Canada from the United States
 - Phone operator considers them one community for promotional offers
- Other community names: *group*, *cluster*, *cohesive subgroup*, or *module*

Examples of Explicit Social Media Communities

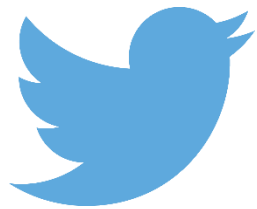


Facebook has groups and communities. Users can

- post messages and images,
- can comment on other messages,
- can like posts, and
- can view activities of other users



In Google+, Circles represent communities



In Twitter, communities form as lists.

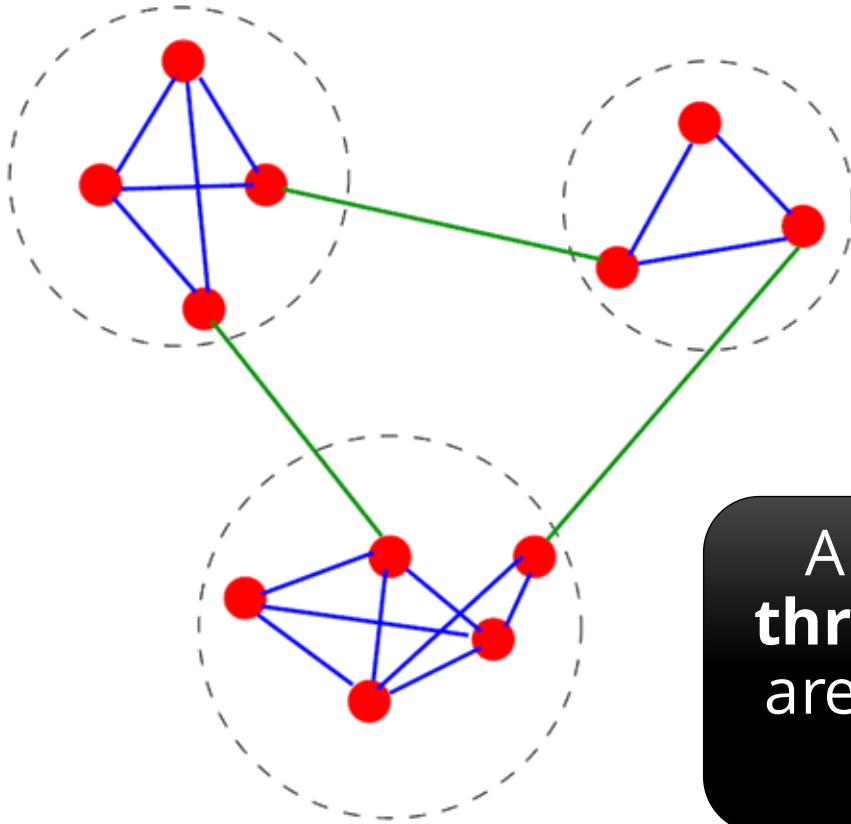
- Users join lists to receive information in the form of tweets



LinkedIn provides *Groups* and *Associations*.

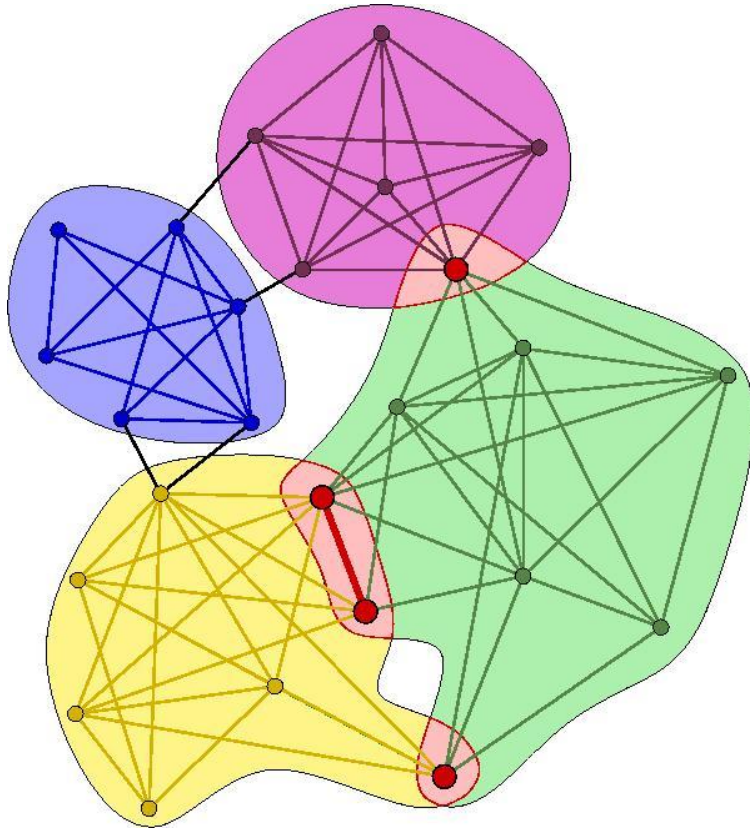
- Users can join professional groups where they can post and share information related to the group

Finding Implicit Communities: An Example

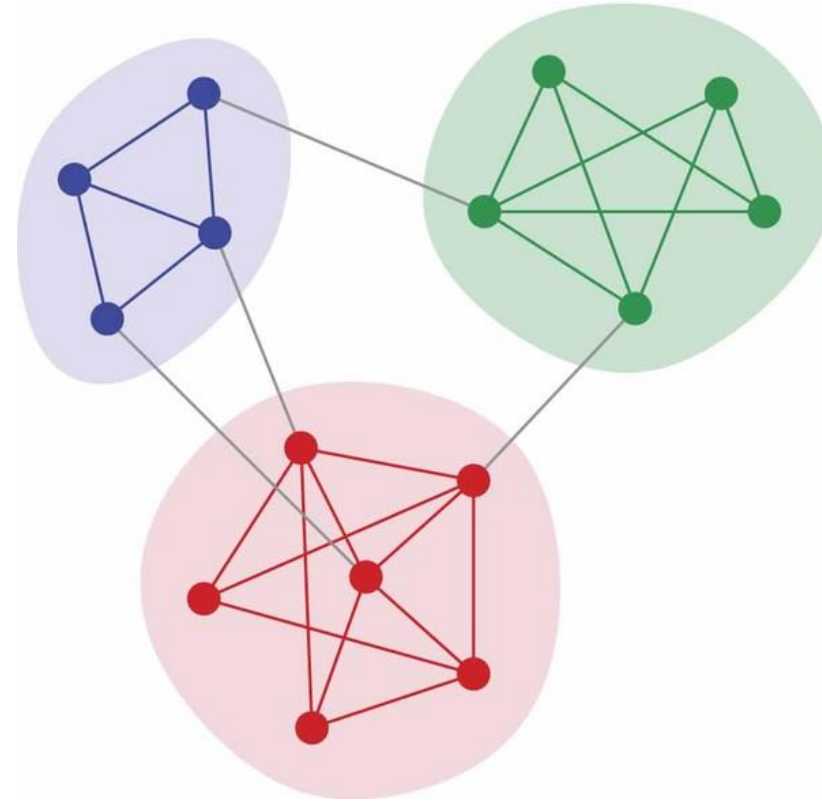


A simple graph in which **three** implicit communities are found, enclosed by the dashed circles

Overlapping vs. Disjoint Communities



Overlapping Communities



Disjoint Communities

Implicit communities in other domains

Protein-protein interaction networks

- Communities are likely to group proteins having the same specific function within the cell

World Wide Web

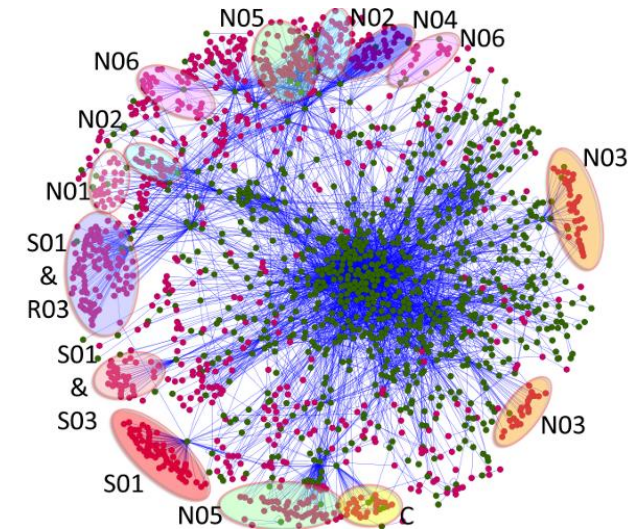
- Communities may correspond to groups of pages dealing with the same or related topics

Metabolic networks

- Communities may be related to functional modules such as cycles and pathways

Food webs

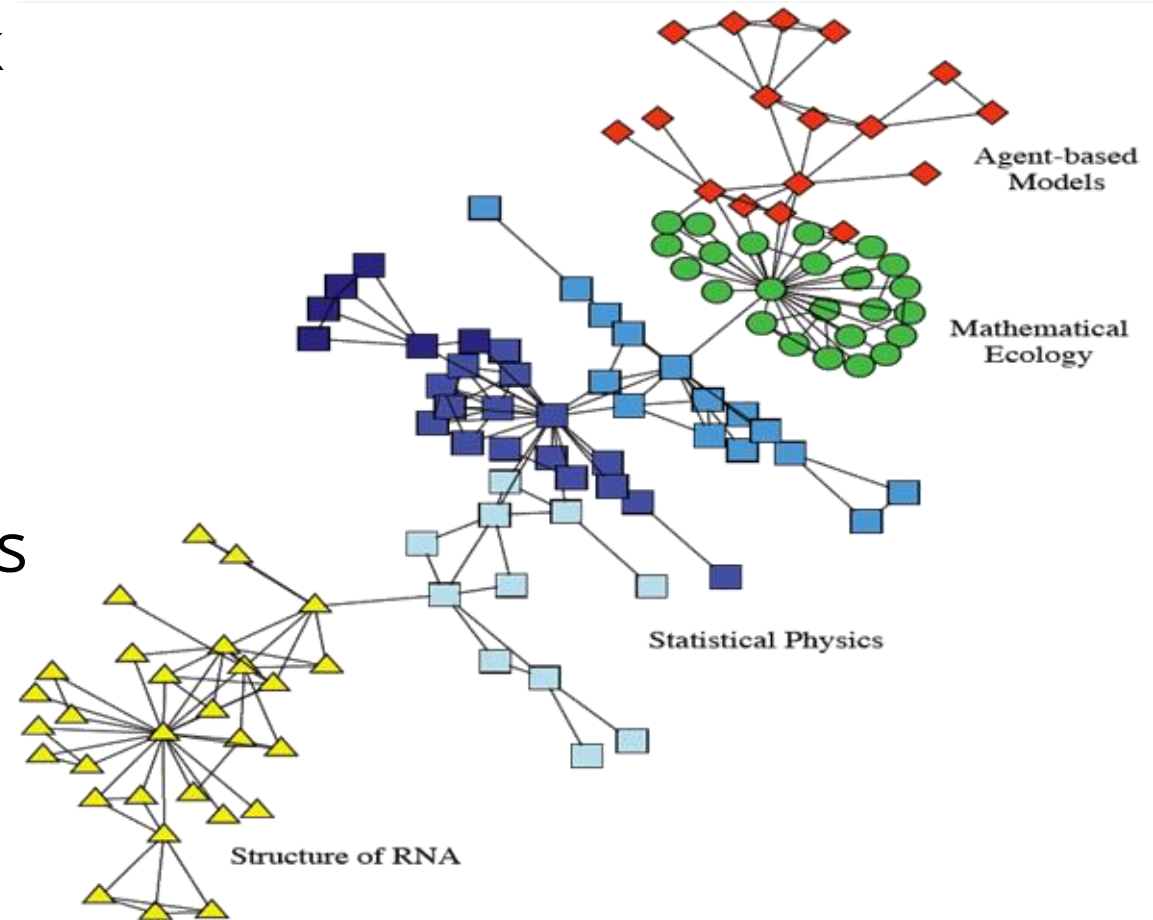
- Communities may identify compartments



Real-world Implicit Communities

Collaboration network
between scientists
working at the
Santa Fe Institute.

The colors indicate
high level communities
and correspond to
research divisions of
the institute



What is Community Analysis?

- **Community detection**
 - Discovering implicit communities
- **Community evolution**
 - Studying temporal evolution of communities
- **Community evaluation**
 - Evaluating Detected Communities

Community Detection

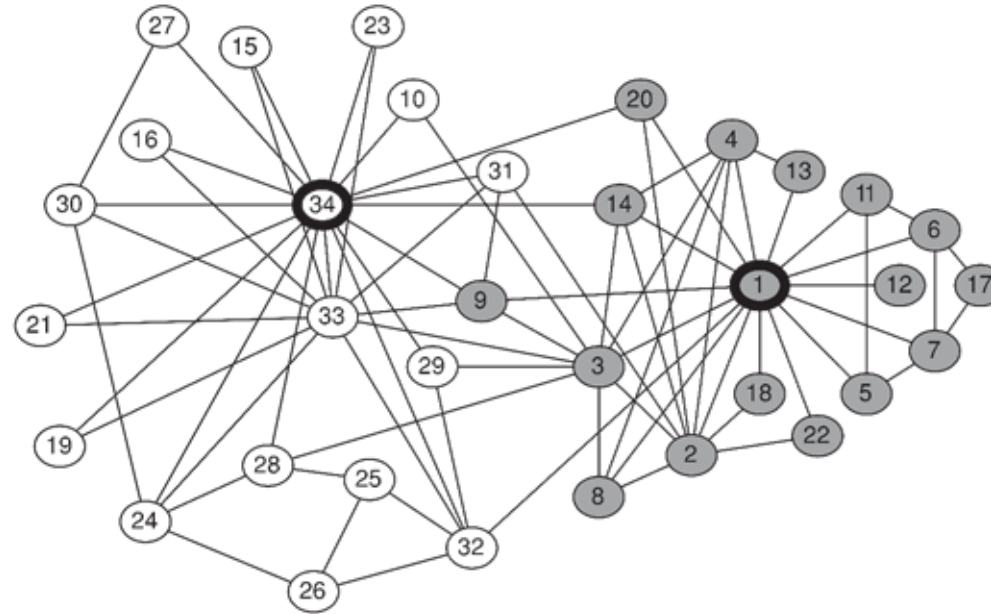
What is community detection?

- The process of finding clusters of nodes (*"communities"*)
 - With **Strong** internal connections and
 - **Weak** connections between different communities
- Ideal decomposition of a large graph
 - Completely disjoint communities
 - There are no interactions between different communities.
- In practice,
 - find community partitions that are maximally decoupled.

Why Detecting Communities is Important?

Zachary's karate club

Interactions between
34 members of a
karate club
for over two years



- The club members split into two groups (**gray** and **white**)
- Disagreement between the administrator of the club (node **34**) and the club's instructor (node **1**),
- The members of one group left to start their own club

**The same communities can be
found using community detection**

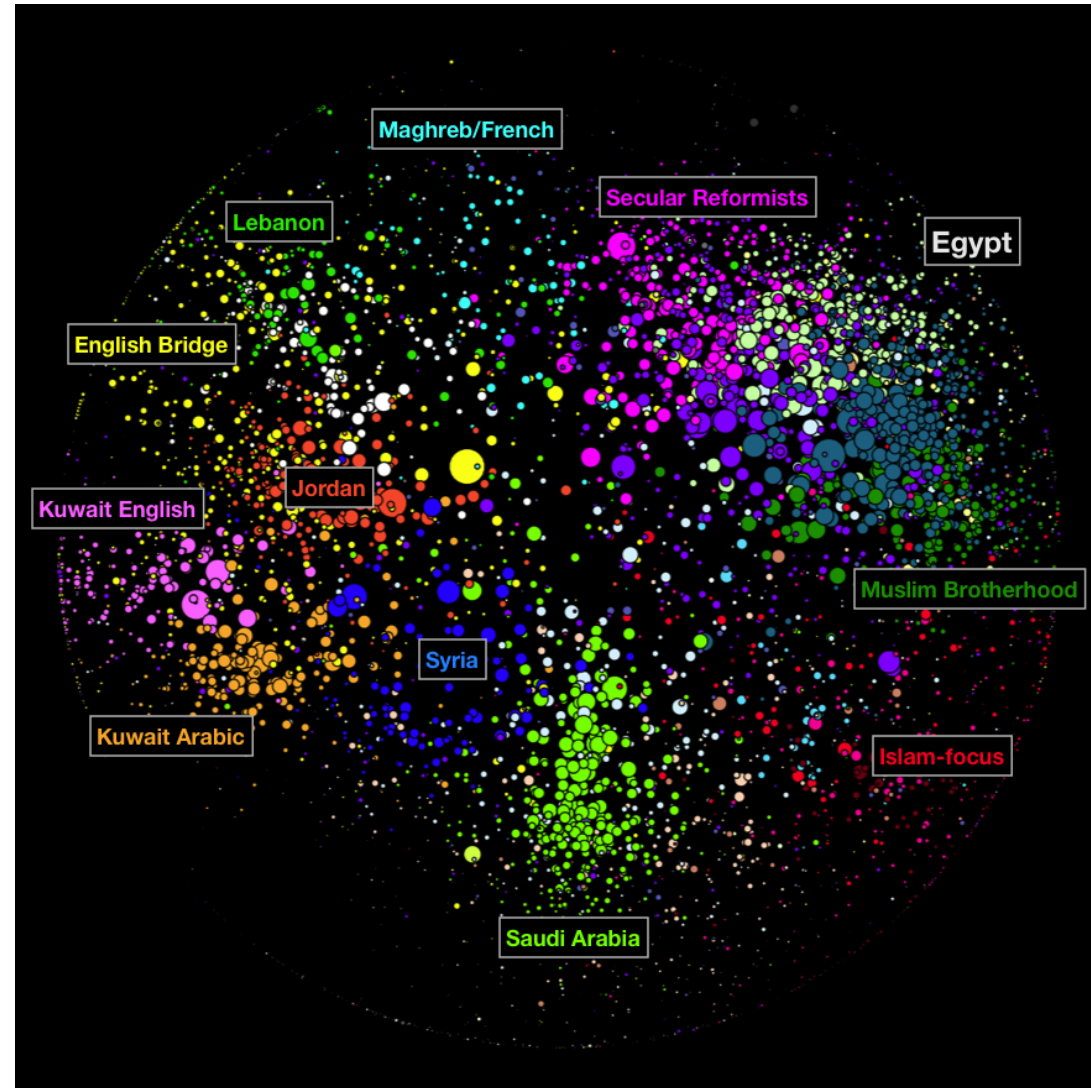
Why Community Detection?

Network Summarization

- A community can be considered as a summary of the whole network
- Easier to visualize and understand

Preserve Privacy

- [Sometimes] a community can reveal some properties without releasing the individuals' privacy information.



Community Detection vs. Clustering

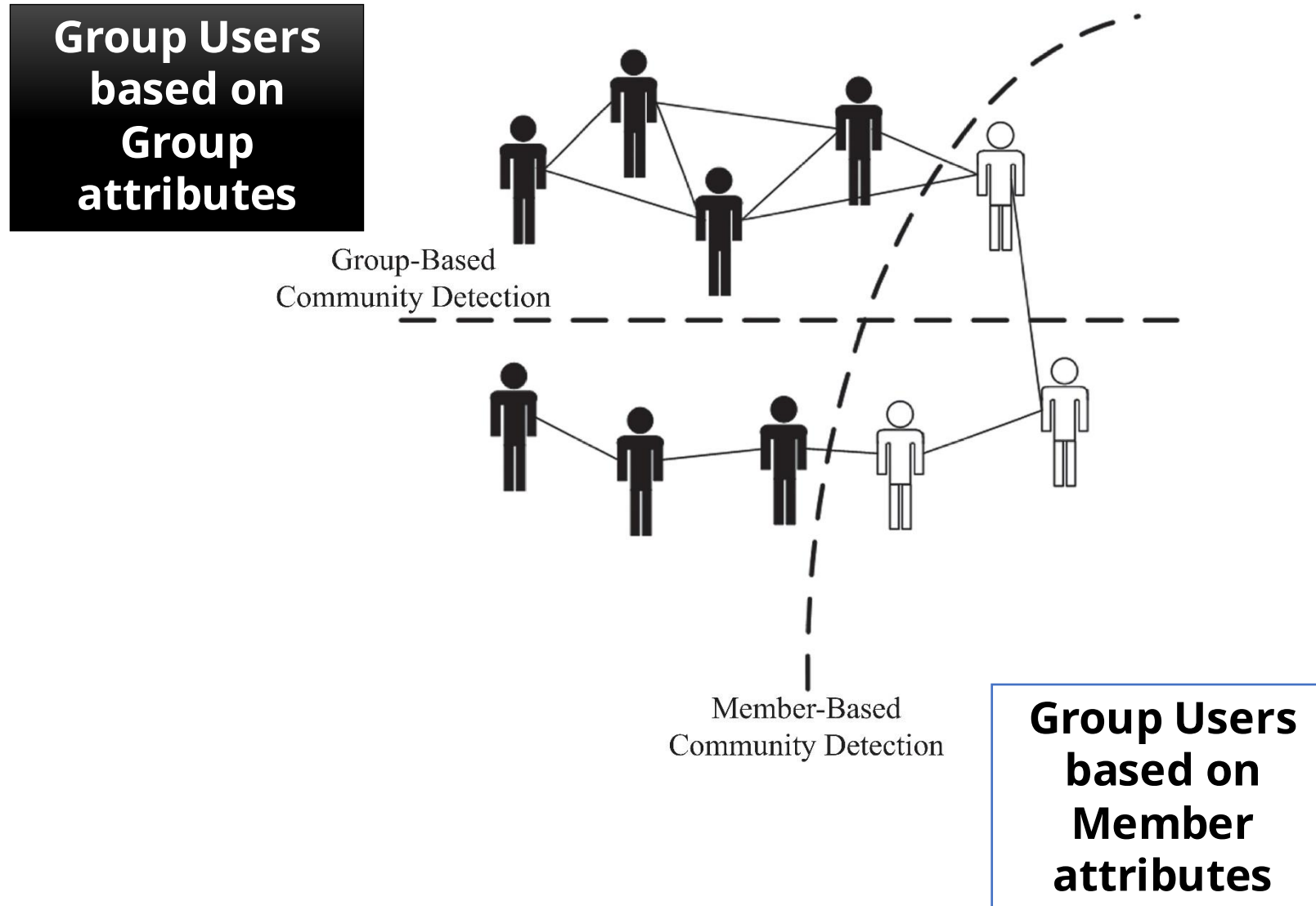
Clustering

- Data is often non-linked (matrix rows)
- Clustering works on the distance or similarity matrix, e.g., k -means.
- If you use k -means with adjacency matrix rows, you are only considering the ego-centric network

Community detection

- Data is linked (a graph)
- Network data tends to be “discrete”, leading to algorithms using the graph property directly
 - k -clique, quasi-clique, or edge-betweenness

Community Detection Algorithms



Community Detection Algorithms

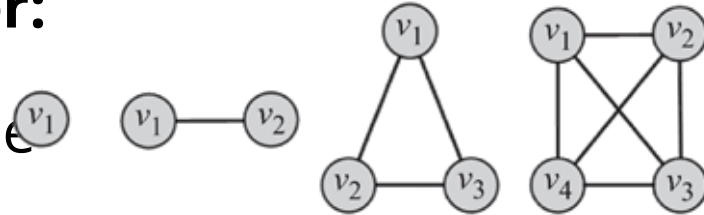
1. Member-Based Community Detection

Basis	Technique / Algorithm	Simple idea
Degree	Clique detection	Members having strong/direct connections form a group
Reachability	k-clique, k-club, k-clan	Members that can reach each other within certain limits form a group
Similarity	Jaccard / Cosine Similarity	Members with similar characteristics are grouped together

A. Node Degree

Most common subgraph searched for:

- **Clique:** a maximum complete subgraph in which all nodes inside the subgraph adjacent to each other



Find communities by searching for

1. **The maximum clique:**
the one with the largest number of vertices, or
2. **All maximal cliques:**
cliques that are not subgraphs of a larger clique; i.e., cannot be further expanded

To overcome this, we can

- I. Brute Force
- II. Relax cliques
- III. Use cliques as the core for larger communities

Both problems are NP-hard

I. Brute-Force Method

Can find all the maximal cliques in the graph

For each vertex v_x , we find the maximal clique that contains node v_x

Algorithm 1 Brute-Force Clique Identification

Require: Adjacency Matrix A , Vertex v_x

```
1: return Maximal Clique  $C$  containing  $v_x$ 
2: CliqueStack =  $\{\{v_x\}\}$ , Processed =  $\{\}$ ;
3: while CliqueStack not empty do
4:    $C = \text{pop}(\text{CliqueStack})$ ; push(Processed,  $C$ );
5:    $v_{last} = \text{Last node added to } C$ ;
6:    $N(v_{last}) = \{v_i | A_{v_{last}, v_i} = 1\}$ .
7:   for all  $v_{temp} \in N(v_{last})$  do
8:     if  $C \cup \{v_{temp}\}$  is a clique then
9:       push(CliqueStack,  $C \cup \{v_{temp}\}$ );
10:    end if
11:  end for
12: end while
13: Return the largest clique from Processed
```

Impractical for large networks:

- For a complete graph of only 100 nodes, the algorithm will generate at least $2^{99} - 1$ different cliques starting from any node in the graph

Enhancing the Brute-Force Performance

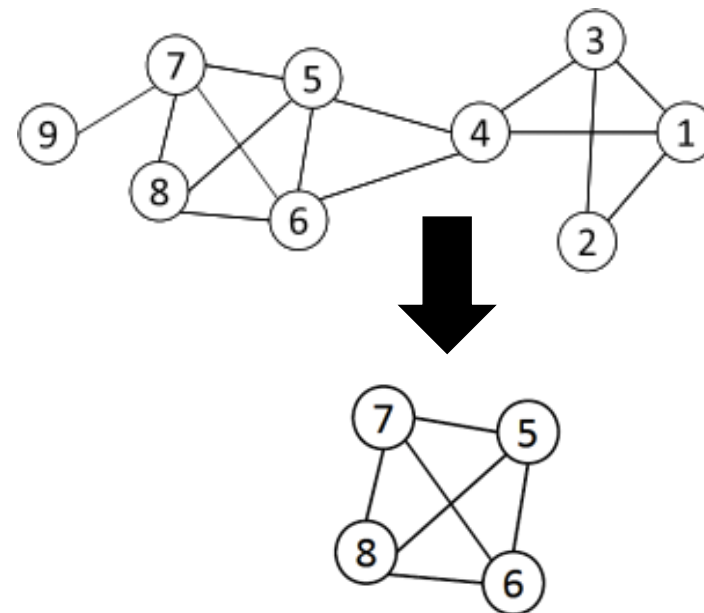
[Systematic] Pruning can help:

- When searching for cliques of size k or larger
- If the clique is found, each node should have a degree equal to or more than $k - 1$
- We can first prune all nodes (and edges connected to them) with degrees less than $k - 1$
 - More nodes will have degrees less than $k - 1$
 - Prune them recursively
- For large k , many nodes are pruned as social media networks follow a power-law degree distribution

Maximum Clique: Pruning...

Example. to find a clique ≥ 4 ,
remove all nodes with degree
 $\leq (4 - 1) - 1 = 2$

- Remove nodes 2 and 9
- Remove nodes 1 and 3
- Remove node 4



Even with pruning, cliques are less desirable

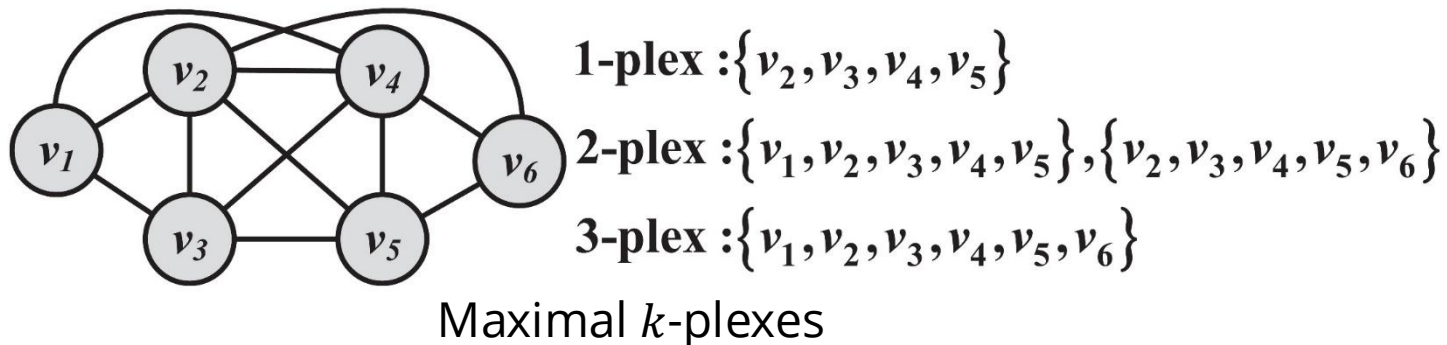
- Cliques are **rare**
- A clique of 1000 nodes, has $999 \times 1000 / 2$ edges
- **A single edge removal** destroys the clique
- That is less than 0.0002% of the edges!

II. Relaxing Cliques

- **k -plex**: a set of vertices V in which we have

$$d_v \geq |V| - k, \forall v \in V$$

- d_v is the degree of v in the induced subgraph
 - Number of nodes from V that are connected to v
- Clique of size k is a 1-plex
- Finding the maximum k -plex: **NP-hard**
 - In practice, relatively easier due to smaller search space.



More Cliques Relaxing...

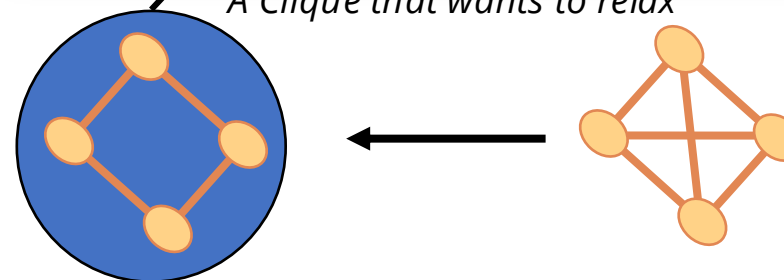
- **k -core**: a maximal connected subgraph in which all vertices have degree at least k
 - Difference with k -plex?
- **k -shell**: nodes that are part of the k -core, but are not part of the $(k + 1)$ -core.

Questions

- 0-core?
- 0-shell?
- 1-core?
- k -cores of the complete graph?



A Clique that wants to relax



A 2-plex or a 2-core?

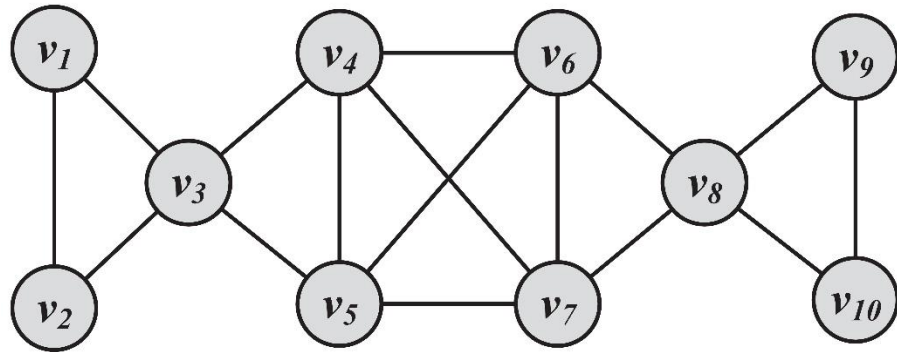
k -core = minimum k connections
 k -plex = almost a clique
 k -shell = layer between k -core and $(k+1)$ -core

III. Using Cliques as a seed of a Community

Clique Percolation Method (CPM)

- Uses cliques as seeds to find larger communities
 - CPM finds overlapping communities
-
- **Input**
 - A parameter k , and a network
 - **Procedure**
 - Find out all cliques of size k in the given network
 - Construct a clique graph.
 - Two cliques are adjacent if they share $k - 1$ nodes
 - Each connected components in the clique graph form a community

Clique Percolation Method: Example



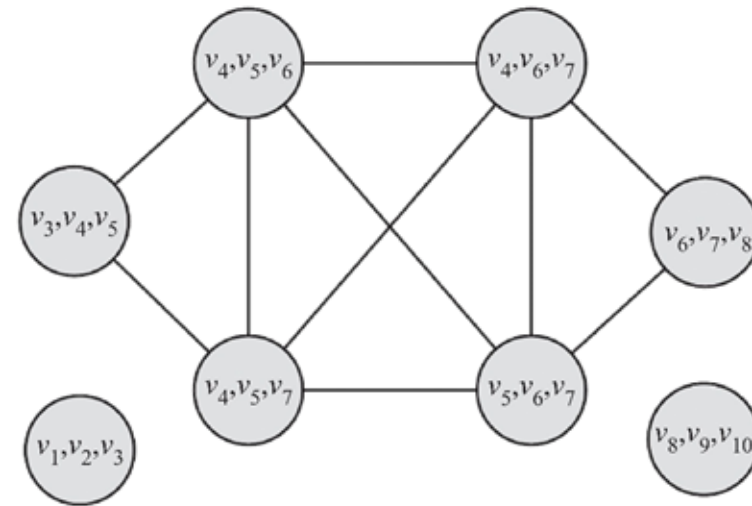
(a) Graph

Cliques of size 3:

$\{v_1, v_2, v_3\}$, $\{v_3, v_4, v_5\}$,
 $\{v_4, v_5, v_6\}$, $\{v_4, v_5, v_7\}$,
 $\{v_4, v_6, v_7\}$, $\{v_5, v_6, v_7\}$,
 $\{v_6, v_7, v_8\}$, $\{v_8, v_9, v_{10}\}$

Communities:

$\{v_1, v_2, v_3\}$,
 $\{v_8, v_9, v_{10}\}$,
 $\{v_3, v_4, v_5, v_6, v_7, v_8\}$



(b) CPM Clique Graph

B. Node Reachability

The two extremes

Nodes are assumed to be in the same community

1. If there is a path between them (regardless of the distance) or
2. They are so close as to be immediate neighbors.

How? Find using BFS/DFS

Challenge: most nodes are in one community (giant component)

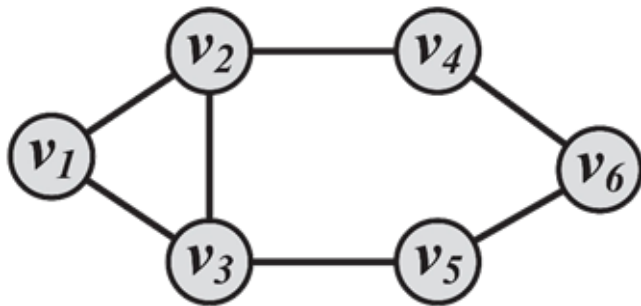
How? Finding Cliques

Challenge: Cliques are challenging to find and are rarely observed

Solution: find communities that are in between **cliques** and **connected components** in terms of connectivity and have small shortest paths between their nodes

Special Subgraphs

1. **k -Clique**: a **maximal** subgraph in which the largest shortest path distance between any nodes is less than or equal to k
2. **k -Club**: follows the same definition as a k -clique
 - **Additional Constraint**: nodes on the shortest paths should be part of the subgraph (i.e., diameter)
3. **k -Clan**: a k -clique where for all shortest paths within the subgraph the distance is equal or less than k .
 - All k -clans are k -cliques, but not vice versa.



2-cliques : $\{v_1, v_2, v_3, v_4, v_5\}, \{v_2, v_3, v_4, v_5, v_6\}$

2-clubs : $\{v_2, v_3, v_4, v_5, v_6\}, \{v_1, v_2, v_3, v_4\}, \{v_1, v_2, v_3, v_5\}$

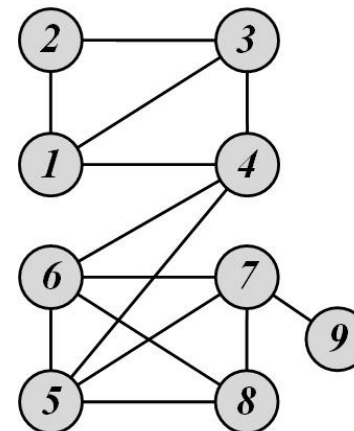
2-clans : $\{v_2, v_3, v_4, v_5, v_6\}$

C. Node Similarity

- Similar (or most similar) nodes are assumed to be in the same community.
 - A classical clustering algorithm (e.g., k -means) is applied to node similarities to find communities.
- Node similarity can be defined
 - Using the similarity of node neighborhoods (**Structural Equivalence**) – Ch. 3
 - Similarity of social circles (**Regular Equivalence**) – Ch. 3

Structural equivalence: two nodes are structurally equivalent iff. they are connecting to the same set of actors

*Nodes 1 and 3 are
structurally equivalent,
So are nodes 5 and 7.*



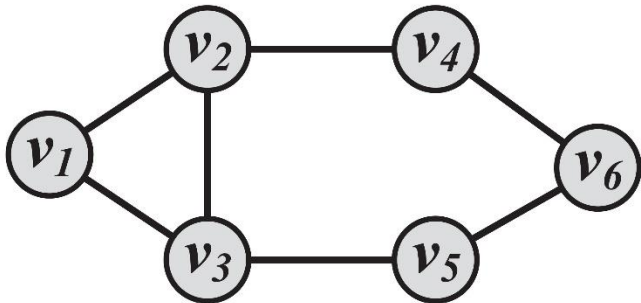
Node Similarity (Structural Equivalence)

Jaccard Similarity

$$\sigma_{\text{Jaccard}}(v_i, v_j) = \frac{|N(v_i) \cap N(v_j)|}{|N(v_i) \cup N(v_j)|}$$

Cosine similarity

$$\sigma_{\text{Cosine}}(v_i, v_j) = \frac{|N(v_i) \cap N(v_j)|}{\sqrt{|N(v_i)| |N(v_j)|}}$$



$$\sigma_{\text{Jaccard}}(v_2, v_5) = \frac{|\{v_1, v_3, v_4\} \cap \{v_3, v_6\}|}{|\{v_1, v_3, v_4, v_6\}|} = 0.25$$

$$\sigma_{\text{Cosine}}(v_2, v_5) = \frac{|\{v_1, v_3, v_4\} \cap \{v_3, v_6\}|}{\sqrt{|\{v_1, v_3, v_4\}| |\{v_3, v_6\}|}} = 0.40$$

Community Detection Algorithms

2. Group-Based Community Detection

Desired community

Balanced

Robust

Modular

Dense

Hierarchical

Example algorithm/technique

Spectral Clustering

k-edge / k-vertex connectivity

Modularity Maximization

Quasi-cliques

Girvan–Newman Algorithm

Community Detection

- **Community Detection** is a crucial task in network science, aimed at **identifying groups of nodes** (or vertices) that are more densely connected to each other than to the rest of the network.
- These groups are called **communities** or **clusters**, and community detection helps in understanding the structure and functionality of complex networks.

Why is Community Detection Important?

- **Social Networks:**

- Communities can represent groups of friends, professional networks, or people with shared interests.

- **Biological Networks:**

- In protein interaction networks, communities might indicate groups of proteins that work together to perform a specific function.

- **Information Networks:**

- In citation or co-authorship networks, communities represent closely related topics or groups of researchers working together.

- **Recommendation Systems:**

- Communities can reveal groups of users with similar preferences, helping improve recommendation algorithms.

Key Concepts in Community Detection:

1.Community:

- A set of nodes that have denser connections within the group than with the rest of the network.
- There is no strict definition of a community, but many algorithms aim to maximize intra-community edges while minimizing inter-community edges.

2.Overlapping vs. Non-Overlapping Communities:

- **Non-overlapping:** Each node belongs to exactly one community (e.g., traditional clustering).
- **Overlapping:** A node can belong to multiple communities, as is common in real-world networks (e.g., a person can belong to multiple social circles).

3.Hierarchical Communities:

- Some networks exhibit hierarchical community structures, where communities can contain smaller sub-communities.

Methods for Community Detection:

- **Clique Percolation Method (CPM)**

- Identifies overlapping cliques of a given size and connects them through shared vertices.
- Communities are groups of cliques that share at least one node.
- Example: Co-authorship networks where researchers belong to overlapping research groups.

Methods for Community Detection:

- **Clique Percolation Method**

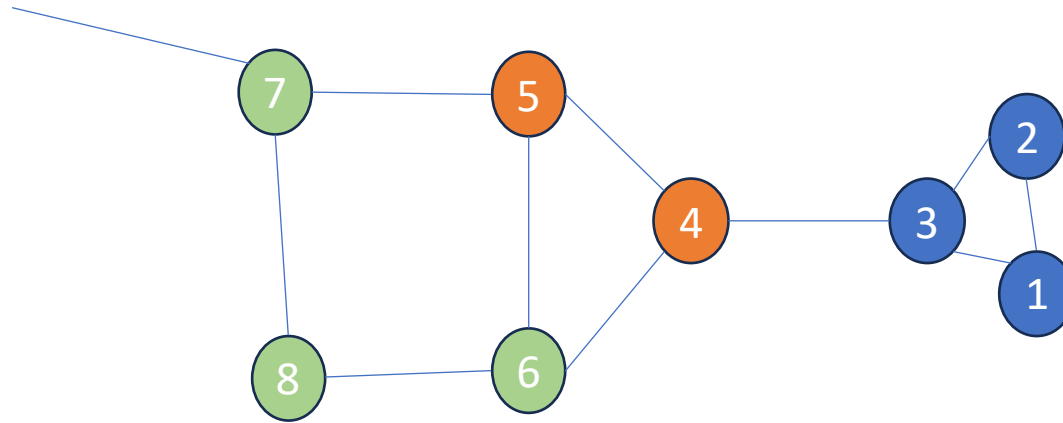
- **How it works:**

- CPM detects communities by identifying **k-cliques** (fully connected subgraphs of k vertices) and forming communities by connecting cliques that share $k-1$ vertices. Communities emerge as groups of overlapping cliques.

Methods for Community Detection:

- **Clique Percolation Method**

- **Example:** In a **collaboration network** (e.g., co-authorship networks), CPM could detect research groups where researchers frequently co-author papers. A clique of size 3 might represent a group of three authors who all wrote a paper together. If other researchers share connections (i.e., worked with at least two of the three), they would be part of a larger community formed by overlapping cliques.



Methods for Community Detection:

- **Clique Percolation Method**

- **Strength:** It allows for **overlapping communities**, meaning an author could belong to multiple research groups (cliques).

Methods for Community Detection:

- **Clique Percolation Method**

```
import networkx as nx

G = nx.Graph()

G.add_edges_from([
    ("A", "B"),
    ("A", "C"),
    ("B", "C"),    # Clique A-B-C

    ("C", "D"),
    ("C", "E"),
    ("D", "E")    # Clique C-D-E
])

communities = nx.community.k_clique_communities(G, k=3)

for i, community in enumerate(communities, 1):
    print("Community", i, ":", sorted(community))
```

Community 1 : ['A', 'B', 'C']

Community 2 : ['C', 'D', 'E']

Methods for Community Detection:

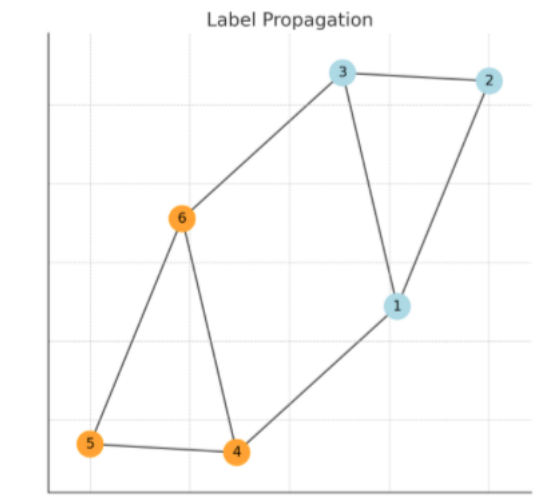
- **Label Propagation:**

- Simple and fast method that spreads labels among neighboring nodes.
- Detects communities by propagating labels iteratively.
- Example: Twitter networks identifying users frequently interacting
- **How it works:**
 - Initially, each node is assigned a unique label. Each node then updates its label to match the most frequent label among its neighbors, continuing this process until a stable set of labels is reached.

Methods for Community Detection:

- **Label Propagation:**

- **Example:** In a **Twitter network**, where nodes are users and edges represent interactions (like retweets), label propagation can be used to detect communities of users who often interact with each other. Over time, the labels will converge, and users who frequently engage with the same content will form a community.
- Label propagation is a **simple and fast algorithm** for large networks. It propagates the most frequent labels among neighbors, eventually leading to stable communities.
- This method works well for dynamic and evolving networks, as it efficiently finds community structures by relying on neighborhood consensus.



Methods for Community Detection:

- **Label Propagation:**
 - **Strength:** Simple, fast, and scalable for very **large dynamic networks**.

Methods for Community Detection:

- **Label Propagation:**

```
Pride 1 : ['Lion_A', 'Lion_B', 'Lion_C']  
Pride 2 : ['Lion_D', 'Lion_E', 'Lion_F']
```

```
import networkx as nx  
  
G = nx.Graph()  
  
G.add_edges_from([  
    ("Lion_A", "Lion_B"),  
    ("Lion_A", "Lion_C"),  
    ("Lion_B", "Lion_C"),  
  
    ("Lion_C", "Lion_D"),  
  
    ("Lion_D", "Lion_E"),  
    ("Lion_E", "Lion_F"),  
    ("Lion_D", "Lion_F")  
)  
  
communities = nx.community.label_propagation_communities(G)  
  
for i, community in enumerate(communities, 1):  
    print("Pride", i, ":", sorted(community))
```

Label propagation works like:

"Look at your neighbors and follow the group that dominates around you."

It is particularly attractive for **large networks** because it is relatively simple and fast.

Methods for Community Detection:

- **Modularity Maximization**

- **Modularity:** This measures the quality of a division of the network into communities. Higher modularity means the edges within communities are denser than the edges between different communities.

Methods for Community Detection:

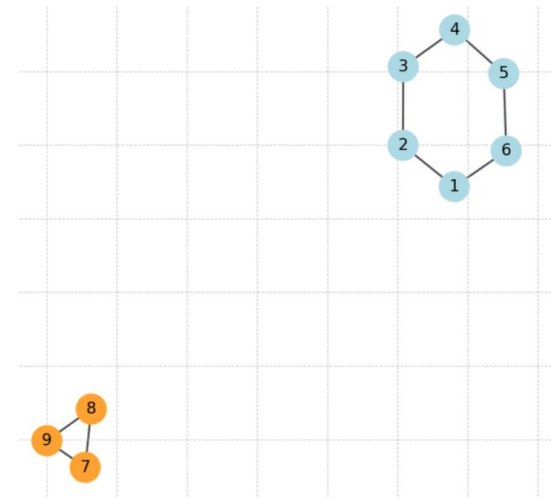
- **Modularity Maximization**

- Divides the network into non-overlapping communities.
- Maximizes the modularity score to ensure dense intra-community connections.
- Example: Social media networks (e.g., Facebook) detecting friendship circles.

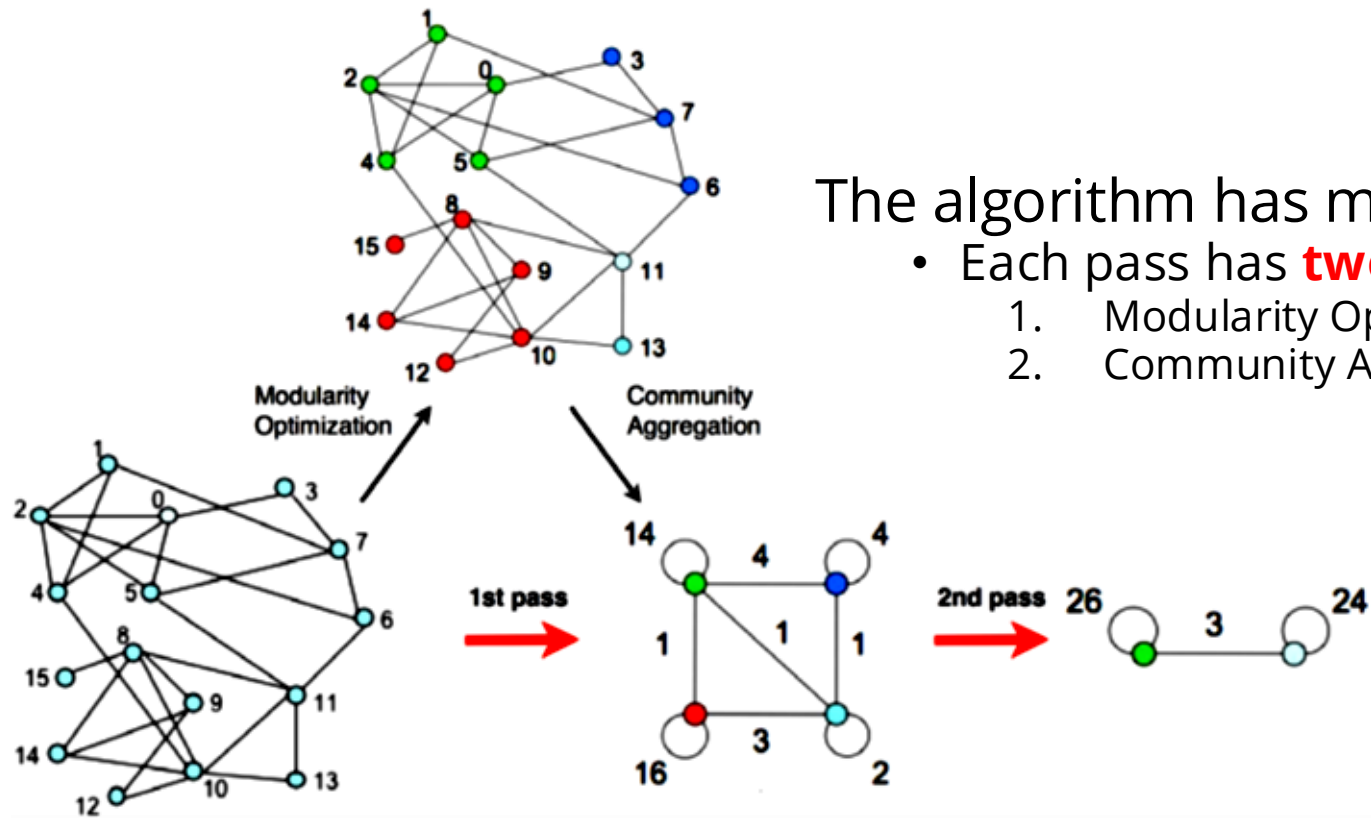
Methods for Community Detection:

- **Modularity Maximization**

- **Example:** In **social media networks** like Facebook, modularity maximization can be used to detect friendship circles. Friends within the same circle (community) will likely have more connections to each other than to people outside their circle.
- The **Louvain algorithm** is often used for this task, which scales well with large networks.



Louvain Method



The algorithm has multiple passes

- Each pass has **two phases**
 1. Modularity Optimization
 2. Community Aggregation

Image from

Blondel, Vincent D., Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. "Fast unfolding of communities in large networks." *Journal of statistical mechanics: theory and experiment* 2008, no. 10 (2008): P10008.

Louvain Method

Start with a weighted network where all nodes are in their own communities (i.e., n communities)

First Phase:

- For each node v_i ,
 - For all neighbors $v_j \in N(v_i)$:
 - compute the modularity gain if v_i is removed from its community and placed in the community of v_j .
 - Find the community with the maximum modularity gain
 - If the maximum gain is positive, remove v_i from its community, and place it in that community
 - If no positive gain, do not change communities
- Repeat until no node changes its community

Important Points about Phase I

- A point can be considered multiple times
- A Local Minima of modularity maximization is achieved in phase I
- Phase I is order dependent
 - The modularity achieved is more or less stable and is less dependent on the initial order
 - The computation time depends on the initial order.



Second Phase:

- Build a new network
 - Nodes are communities
 - Edges are the edges between nodes in the corresponding communities (weights are sum of the weights)
 - Self-loops represent edges within the community
- The algorithm creates hierarchies of communities
- It usually ends in less than 10 passes
- It is seems to be an $O(n \log n)$ algorithm

Methods for Community Detection:

- **Modularity Maximization**
 - **Strength:** This method excels in finding **non-overlapping communities**, meaning each user will belong to exactly one community.

Methods for Community Detection:

- **Modularity Maximization**

```
Community 1 : ['Ahmed', 'Ali', 'Sara', 'Usman']  
Community 2 : ['Ayesha', 'Bilal', 'Hina', 'Zain']
```

```
import networkx as nx  
  
G = nx.Graph()  
  
G.add_edges_from([  
    # Community 1  
    ("Ali", "Ahmed"),  
    ("Ali", "Sara"),  
    ("Ahmed", "Sara"),  
    ("Ahmed", "Usman"),  
    ("Sara", "Usman"),  
    ("Ali", "Usman"),  
  
    # Community 2  
    ("Bilal", "Hina"),  
    ("Bilal", "Ayesha"),  
    ("Hina", "Ayesha"),  
    ("Hina", "Zain"),  
    ("Ayesha", "Zain"),  
    ("Bilal", "Zain"),  
  
    # Small connection between communities  
    ("Usman", "Bilal")  
)  
  
communities = nx.community.greedy_modularity_communities(G)  
  
for i, community in enumerate(communities, 1):  
    print("Community", i, ":", sorted(community))
```

Methods for Community Detection:

- **Spectral Clustering**

- **How it works:** Spectral clustering uses the eigenvalues (or spectrum) of matrices
- **Laplacian matrix** of the graph to detect community structures.

The graph Laplacian is:

$$L = D - A$$

where:

- A = Adjacency Matrix
- D = Degree Matrix
- L = Laplacian Matrix

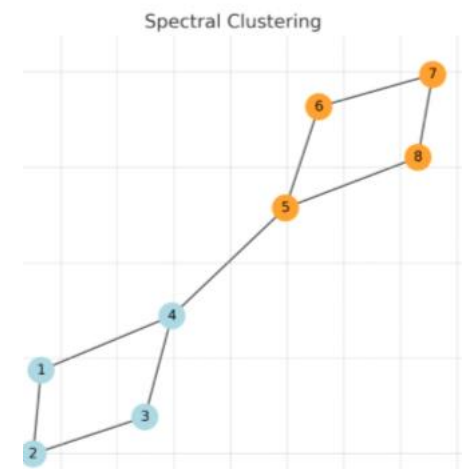
- It effectively partitions the network based on these eigenvalues.
- Uses eigenvalues of the graph Laplacian to partition the network.
- Effective for partitioning well-separated communities.
- Example: Political networks identifying factions or voting patterns.

$$\det(L - \lambda I) = 0$$

Methods for Community Detection:

- **Spectral Clustering**

- **Example:** In a **political network**, where nodes are politicians and edges represent collaborations or shared political affiliations, spectral clustering can help identify groups of politicians with similar voting patterns. These clusters (communities) might reflect political factions or parties.
- The method finds **community structures** by minimizing connections between different clusters while keeping nodes within each cluster highly connected.
- This method is particularly effective when there are **clear, mathematically definable separations** in the graph, such as in this case with two well-defined clusters.



Methods for Community Detection:

- **Spectral Clustering**

- **Strength:** This method can handle **large-scale networks** and can be effective when the network has an underlying mathematical structure.

Methods for Community Detection:

- Spectral Clustering

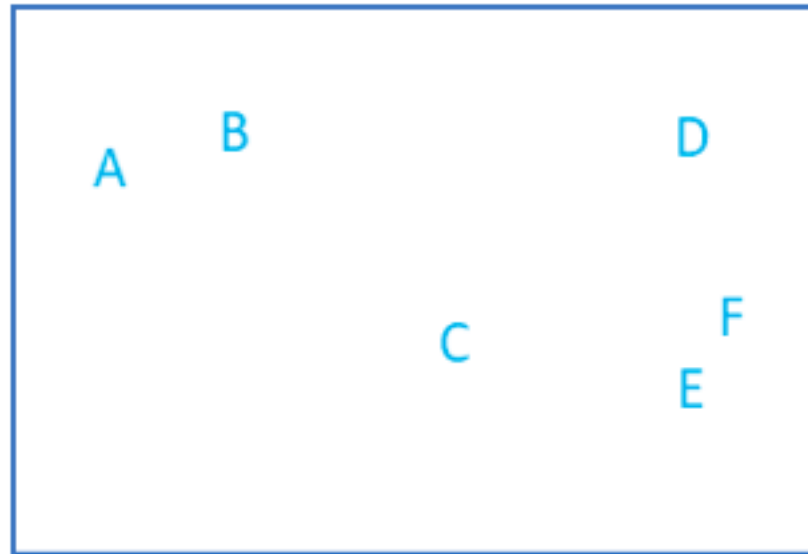
```
Group 1 : ['Lahore', 'Karachi', 'Islamabad', 'Peshawar']  
Group 2 : ['Tokyo', 'Osaka']
```

```
import numpy as np  
from sklearn.cluster import SpectralClustering  
  
cities = [  
    "Lahore",  
    "Karachi",  
    "Islamabad",  
    "Peshawar",  
    "Tokyo",  
    "Osaka"  
]  
  
# Similarity matrix  
similarity = np.array([  
    [1.0, 0.8, 0.7, 0.8, 0.1, 0.1],  
    [0.8, 1.0, 0.7, 0.6, 0.1, 0.1],  
    [0.7, 0.7, 1.0, 0.8, 0.1, 0.1],  
    [0.8, 0.6, 0.8, 1.0, 0.1, 0.1],  
    [0.1, 0.1, 0.1, 0.1, 1.0, 0.9],  
    [0.1, 0.1, 0.1, 0.1, 0.9, 1.0]  
)  
  
model = SpectralClustering(  
    n_clusters=2,  
    affinity="precomputed",  
    random_state=42  
)  
  
labels = model.fit_predict(similarity)  
  
for i in range(2):  
    group = [  
        cities[j]  
        for j in range(len(cities))  
        if labels[j] == i  
    ]  
  
    print("Group", i + 1, ":", group)
```

Methods for Community Detection:

- **Hierarchical Clustering**

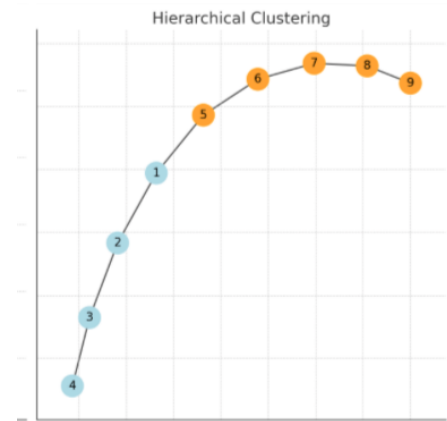
- Creates a hierarchy of communities, visualized through a dendrogram.
- Useful for discovering nested communities at different levels.
- Example: Biological networks grouping proteins by function.
- **How it works:** This method creates a hierarchy of communities, either starting from individual nodes (agglomerative) or from the entire graph (divisive). A dendrogram can be used to visualize this hierarchy.



Methods for Community Detection:

- **Hierarchical Clustering**

- **Example:** In **biological networks**, such as gene-protein interaction networks, hierarchical clustering can identify how proteins are related in terms of function. At one level, you might detect general functional groups (e.g., metabolism-related proteins), and at deeper levels, specific sub-functions within those groups (e.g., proteins involved in glucose metabolism).
- The method can capture **nested community structures**.
- In this case, nodes within each cluster are more closely related to each other.
- Hierarchical clustering is often used to identify **sub-communities** within a larger group.



Methods for Community Detection:

- **Hierarchical Clustering**

- **Strength:** It's good at detecting **hierarchical structures** and nested communities.

Methods for Community Detection

- Hierarchical Clustering

```
Cluster 1 : ['Classical Song', 'Jazz Song']  
Cluster 2 : ['Pop Song', 'Dance Song']  
Cluster 3 : ['Rock Song', 'Metal Song']
```

```
import numpy as np  
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster  
  
songs = [  
    "Rock Song",  
    "Metal Song",  
    "Pop Song",  
    "Dance Song",  
    "Classical Song",  
    "Jazz Song"  
]  
  
# [tempo, energy]  
features = np.array([  
    [140, 0.95], # Rock  
    [150, 0.98], # Metal  
    [120, 0.70], # Pop  
    [125, 0.85], # Dance  
    [70, 0.20], # Classical  
    [90, 0.35]  # Jazz  
)  
  
Z = linkage(features, method="ward")  
  
labels = fcluster(Z, t=3, criterion="maxclust")  
  
for group in sorted(set(labels)):  
    members = [  
        songs[i]  
        for i in range(len(songs))  
        if labels[i] == group  
    ]  
  
    print("Cluster", group, ":", members)
```

Methods for Community Detection:

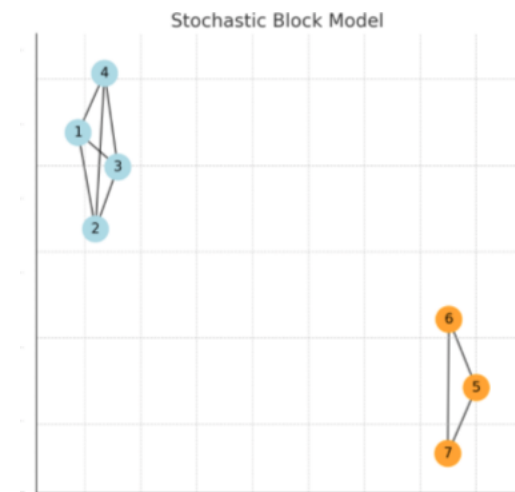
- **Stochastic Block Model**

- Assumes the graph is divided into probabilistic blocks.
- Nodes within the same block have a higher probability of connecting.
- Example: Corporate email networks detecting communication groups.
- **How it works:**
 - SBM assumes that the graph is generated from a block structure, where nodes within the same block (community) have a higher probability of being connected. It uses a probabilistic approach to infer these block structures.

Methods for Community Detection:

- **Stochastic Block Model**

- **Example:** In a **corporate email network**, where nodes represent employees and edges represent email exchanges, SBM can help detect departments or teams within the organization based on communication patterns. Employees within the same department are more likely to email each other, forming a block (community).
- In this method, we generate a **probabilistic model** of the network, where nodes are assigned to different blocks based on connection patterns.



Methods for Community Detection:

- **Stochastic Block Model**

- It's useful in cases where relationships are not deterministic but based on probabilities, such as in **social groups, corporate networks**, or any system where interaction is driven by likelihood rather than certainty.
- **Strength:** SBM is powerful for **discovering hidden structures** in networks where connections are probabilistic rather than deterministic.

Methods for Community Detection:

- **Stochastic Block Model**

```
Total employees: 15
Total communication links: 32
Department 1 : [0, 1, 2, 3, 4]
Department 2 : [5, 6, 7, 8, 9]
Department 3 : [10, 11, 12, 13, 14]
```

Here we **know the groups** because we created the network **using those groups**.

The idea of SBM is that if we were given only the network, we could try to infer the **hidden block/community structure** from the connection patterns.

```
import networkx as nx

sizes = [5, 5, 5]

# Probability of connections
probabilities = [
    [0.8, 0.1, 0.1],
    [0.1, 0.8, 0.1],
    [0.1, 0.1, 0.8]
]

G = nx.stochastic_block_model(
    sizes,
    probabilities,
    seed=10
)

print("Total employees:", G.number_of_nodes())
print("Total communication links:", G.number_of_edges())

for i, block in enumerate(G.graph["partition"], 1):
    print("Department", i, ":", sorted(block))
```

Methods for Community Detection:

- **Info Map**

- Minimizes the description length of random walks to reveal communities.
- **How it works:** Infomap is an information-theoretic approach that minimizes the description length of a random walk on the network. The idea is that a random walker will spend a long time within communities before moving to another community.

Methods for Community Detection:

- **Info Map**

- **Example:** In **web browsing behavior networks**, where nodes represent websites and edges represent transitions between them, Infomap can help identify clusters of websites that users frequently navigate between. For example, a community might consist of a group of news websites, whereas another might be composed of social media platforms.
- **Strength:** It's particularly effective at finding **well-defined communities** in both small and large networks.

Methods for Community Detection:

- Info Map

```
from infomap import Infomap

im = Infomap()

# Area 1
im.add_link("Station_A", "Station_B")
im.add_link("Station_B", "Station_C")
im.add_link("Station_A", "Station_C")

# Area 2
im.add_link("Station_D", "Station_E")
im.add_link("Station_E", "Station_F")
im.add_link("Station_D", "Station_F")

# Connection between areas
im.add_link("Station_C", "Station_D")

im.run()

for node in im.tree:
    if node.is_leaf:
        print(
            node.node_id,
            "-> Community",
            node.module_id
        )
```

```
Station_A -> Community 1
Station_B -> Community 1
Station_C -> Community 1
```

```
Station_D -> Community 2
Station_E -> Community 2
Station_F -> Community 2
```

Infomap essentially asks:

"If information/passengers move through this network, where do they tend to remain?"

Areas with strong internal movement become communities.

Methods for Community Detection:

Summary Table with Examples:

Method	How it Works	Example	Strength
Clique Percolation	Finds k -cliques and links them if they share $k - 1$ vertices	Co-authorship networks: Researchers forming overlapping collaboration groups	Overlapping communities
Modularity Maximization	Maximizes the difference between intra-community and inter-community edges	Social media networks: Detecting friendship circles on platforms like Facebook	Non-overlapping communities
Spectral Clustering	Uses eigenvalues of the graph Laplacian to partition nodes	Political networks: Identifying political factions based on collaboration or voting patterns	Handles large-scale networks
Label Propagation	Nodes adopt the majority label from their neighbors	Twitter networks: Detecting groups of users who frequently interact	Simple, scalable for large dynamic networks
Hierarchical Clustering	Builds a hierarchy of communities, visualized with a dendrogram	Biological networks: Grouping proteins by function and sub-function	Hierarchical communities
Stochastic Block Model	Assumes the network is generated by a probabilistic block structure	Corporate email networks: Detecting communication groups like departments	Reveals hidden structures
Infomap	Minimizes the description length of random walks to reveal community structures	Web browsing networks: Clustering websites that users often navigate between	Good for well-defined communities , large networks

Challenges in Community Detection

- **Scalability:**
 - Many algorithms struggle with large-scale networks.
- **Overlapping Communities:**
 - Complex to detect nodes in multiple groups.
- **Dynamic Networks:**
 - Communities change over time.
- **Evaluation:**
 - No single best way to evaluate detected communities

Real-World Applications

- **Social Networks:**
 - Suggesting groups or friendships (e.g., Facebook).
- **Biological Networks:**
 - Grouping proteins and genes.
- **Marketing:**
 - Identifying customer groups for targeted marketing.
- **Fraud Detection:**
 - Detecting anomalous communities in financial networks.

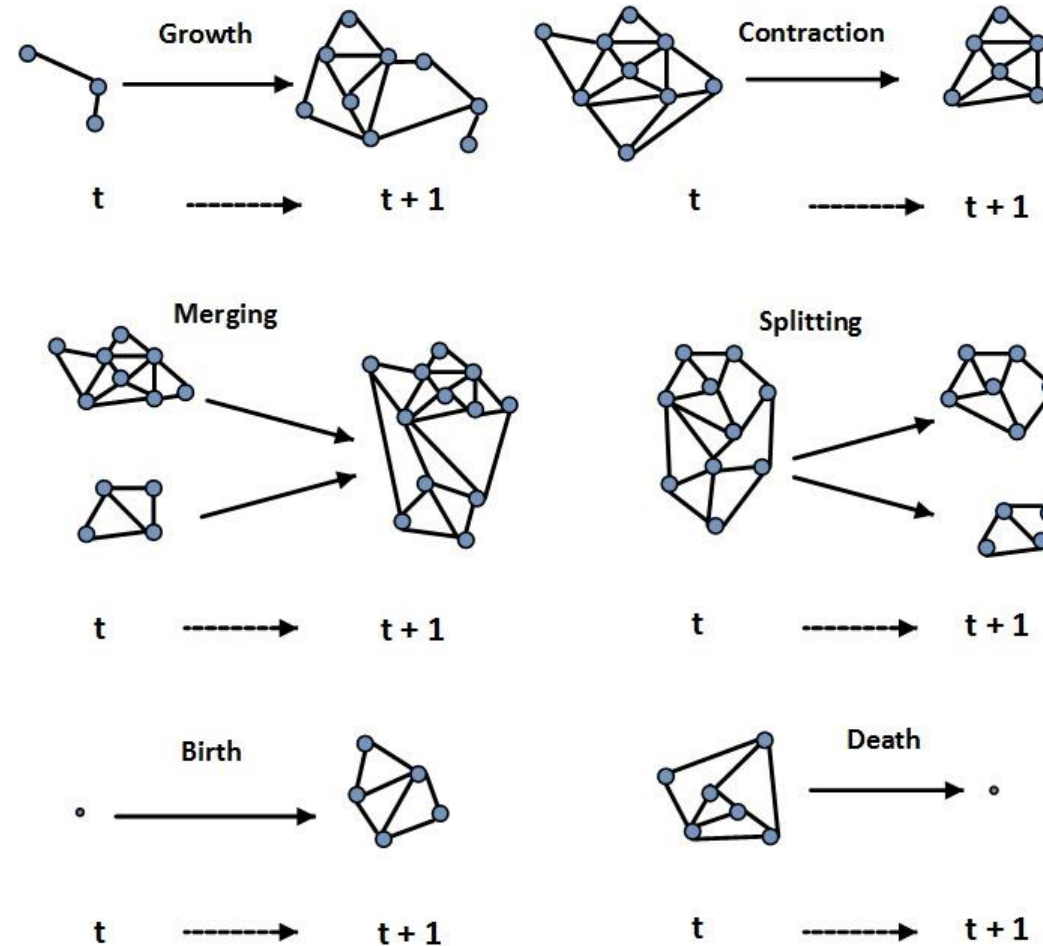
Summary

- Community detection helps understand complex networks' structures.
- Various algorithms detect different types of communities.
- The choice of method depends on the network and community structure.

How Communities Evolve?

Community Evolution

- Communities also expand, shrink, or dissolve in dynamic networks

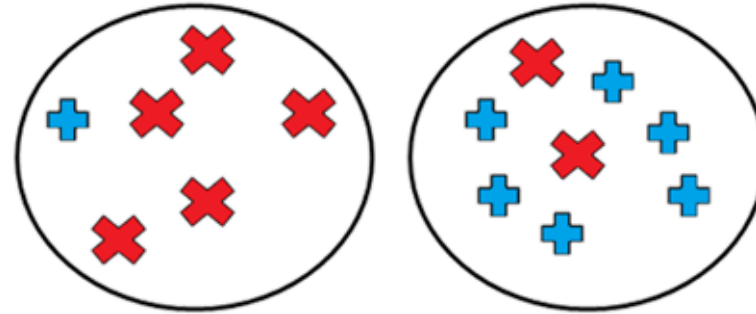


Community Evaluation

Evaluating the Communities

We are given objects of two different kinds (+, ×)

- **The perfect community:** all objects inside the community are of the same type



- **Evaluation with ground truth**
- **Evaluation without ground truth**

Evaluation with Ground Truth

- When ground truth is available
 - We have partial knowledge of what communities should look like
 - We are given the correct community (clustering) assignments
- **Measures**
 - Precision and Recall, or F-Measure

Precision and Recall

$$\text{Precision} = \frac{\text{Relevant and retrieved}}{\text{Retrieved}}$$

$$P = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{\text{Relevant and retrieved}}{\text{Relevant}}$$

$$R = \frac{TP}{TP + FN}$$

True Positive (TP) :

- When similar members are assigned to the same communities
- A **correct** decision.

True Negative (TN) :

- When dissimilar members are assigned to different communities
- A **correct** decision

False Negative (FN) :

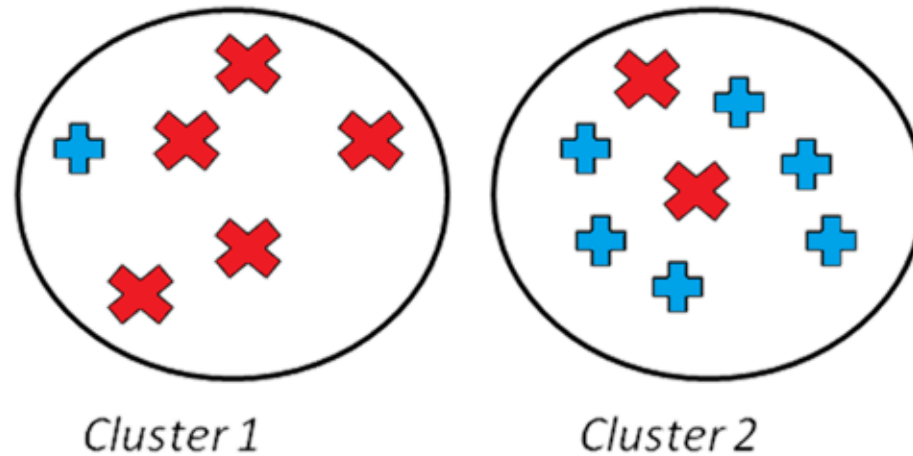
- When similar members are assigned to different communities
- An **incorrect** decision

False Positive (FP) :

- When dissimilar members are assigned to the same communities
- An **incorrect** decision

Precision and Recall: Example

counting individual objects.



Actual \ Predicted

Red (Positive)

Blue (Negative)

Cluster 1 (Positive)

TP = 5

FP = 1

Cluster 2 (Negative)

FN = 2

TN = 6

Precision = $5 / (5 + 1) = 83.33\%$

Recall = $5 / (5 + 2) = 71.43\%$

Precision and Recall: Example

If this is actually a clustering problem

in the context of **K-means/clustering evaluation**, there is an important distinction: **Precision and Recall are not normally calculated simply from "Cluster 1" and "Cluster 2" unless you have ground-truth labels and establish which cluster corresponds to which class.**

For a clustering algorithm, you typically have:

Ground Truth:

Class A: 
Class B: 

Predicted Class

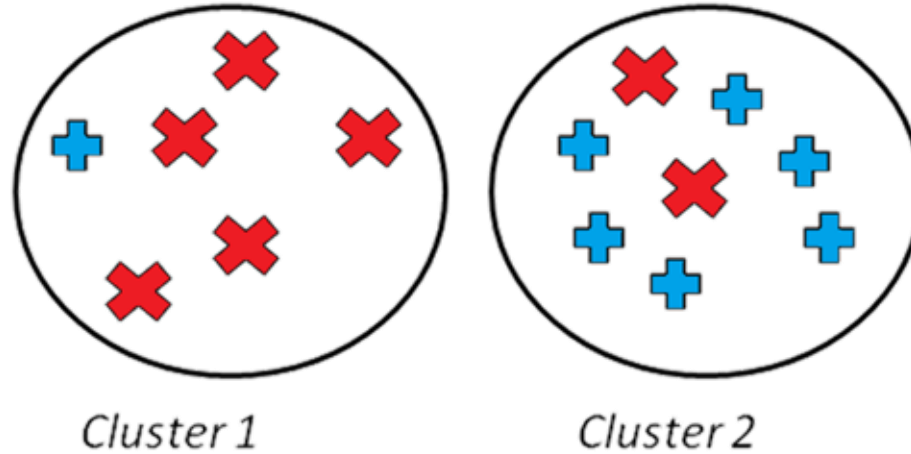
Cluster 1: 
Cluster 2: 

Then you can construct a **confusion matrix** and calculate precision, recall, F1, purity, NMI

community detection is fundamentally about grouping relationships between nodes, rather than assigning each node an independent class.

Precision and Recall: Example

counting **pairs of objects**.



$$TP = \binom{1}{2} + \binom{5}{2} + \binom{6}{2} + \binom{2}{2} = 26$$

$$FP = (5 \times 1) + (6 \times 2) = 17,$$

$$FN = (5 \times 2) + (6 \times 1) = 16,$$

$$TN = (6 \times 5) + (2 \times 1) = 32.$$

$$P = \frac{26}{26+17} = 0.60$$

$$R = \frac{26}{26+16} = 0.61$$

Either P or R measures one aspect of the performance,

- To integrate them into one measure, we can use the harmonic mean of precision of recall

$$F = 2 \cdot \frac{P \cdot R}{P + R}$$

For the example earlier,

$$F = 2 \times \frac{0.6 \times 0.61}{0.6 + 0.61} = 0.60$$

- Pair-based evaluation is useful in community detection because a community is fundamentally about which nodes belong together, so evaluating node pairs directly measures whether the detected partition preserves the true "together vs. separate" relationships.

We can assume the majority of a community represents the community

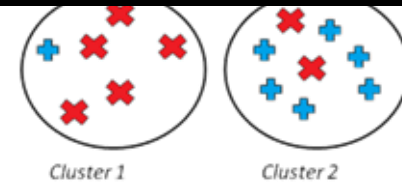
- We use the label of the majority against the label of each member to evaluate the communities

Purity can be easily **tampered** by

- Points being singleton communities (of size 1); or by
- Very large communities

$$Purity = \frac{1}{N} \sum_{i=1} \max_j |C_i \cap L_j|$$

- k : the number of communities
- N : total number of nodes,
- L_j : the set of instances with label j in all communities
- C_i : the set of members in community i



purity is: $\frac{6+5}{14} = 0.78$

Mutual Information

- **Mutual information (MI).** The amount of information that two random variables share.
 - By knowing one of the variables, it measures the amount of uncertainty reduced regarding the others

$$MI = I(H, L) = \sum_{h \in H} \sum_{l \in L} \frac{n_{h,l}}{n} \log \frac{n \cdot n_{h,l}}{n_h n_l}$$

- L and H are labels and found communities;
- n_h and n_l are the number of data points in community h and with label l , respectively;
- $n_{h,l}$ is the number of nodes in community h and with label l ; and n is the number of nodes

Mutual Information

$$n = 14$$

There are two detected communities:

$$H = \{H_1, H_2\}$$

and two actual communities:

$$L = \{L_1, L_2\}$$

The table at the bottom is essentially:

	Actual L_1	Actual L_2	Total
Detected H_1	5	1	6
Detected H_2	2	6	8
Total	7	7	14

$$n_{1,1} = 5$$

$$n_{1,2} = 1$$

$$n_{2,1} = 2$$

$$n_{2,2} = 6$$

$$MI = I(H, L) = \sum_{h \in H} \sum_{l \in L} \frac{n_{h,l}}{n} \log \frac{n \cdot n_{h,l}}{n_h n_l}$$

$$MI = \frac{5}{14} \log \frac{14(5)}{6(7)} + \frac{1}{14} \log \frac{14(1)}{6(7)} \\ + \frac{2}{14} \log \frac{14(2)}{8(7)} + \frac{6}{14} \log \frac{14(6)}{8(7)}$$

$$MI \approx 0.179$$

Normalizing Mutual Information (NMI)

- Mutual information (MI) is unbounded
- To address this issue, we can normalize MI

- How? We know that

$$\begin{aligned} MI &\leq \min(H(L), H(H)), \\ (MI)^2 &\leq H(H)H(L). \\ MI &\leq \sqrt{H(H)} \sqrt{H(L)}. \end{aligned}$$

- $H(\cdot)$ is the entropy function

$$H(L) = - \sum_{l \in L} \frac{n_l}{n} \log \frac{n_l}{n} \quad H(L) = -\frac{7}{14} \log \frac{7}{14} - \frac{7}{14} \log \frac{7}{14}$$

$$H(L) = 0.693$$

$$H(H) = - \sum_{h \in H} \frac{n_h}{n} \log \frac{n_h}{n} \quad H(H) = -\frac{6}{14} \log \frac{6}{14} - \frac{8}{14} \log \frac{8}{14}$$

$$H(H) \approx 0.683$$

Normalized Mutual Information

Normalized Mutual Information

$$NMI = \frac{MI}{\sqrt{H(L)} \sqrt{H(H)}}.$$

$$NMI = \frac{\sum_{h \in H} \sum_{l \in L} n_{h,l} \log \frac{n \cdot n_{h,l}}{n_h n_l}}{\sqrt{(\sum_{h \in H} n_h \log \frac{n_h}{n})(\sum_{l \in L} n_l \log \frac{n_l}{n})}}.$$

We can also define it as

Note that $MI < 1/2(H(H) + H(L))$

$$NMI = \frac{I(H; L)}{\frac{1}{2}(H(L) + H(H))}$$

$$MI = 0.179$$

$$H(L) = 0.693$$

$$H(H) = 0.683$$

$$NMI = \frac{0.179}{\sqrt{(0.693)(0.683)}}$$

y:

$$\sqrt{0.473} = 0.688$$

$$NMI = \frac{0.179}{0.688}$$

$$NMI \approx 0.260$$

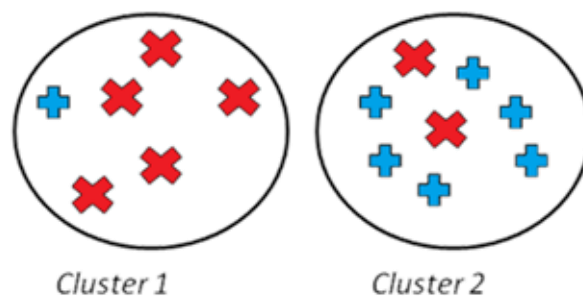
$$26\%$$

Normalized Mutual Information

$$NMI = \frac{\sum_{h,l} n_{h,l} \log \frac{n \cdot n_{h,l}}{n_h n_l}}{\sqrt{(\sum_h n_h \log \frac{n_h}{n})(\sum_l n_l \log \frac{n_l}{n})}}$$

- where l and h are known (with labels) and found communities, respectively
 - n_h and n_l are the number of members in the community h and l , respectively,
 - $n_{h,l}$ is the number of members in community h and labeled l ,
 - n is the size of the dataset
-
- **NMI** values close to one indicate high similarity between communities found and labels
 - Values close to zero indicate high dissimilarity between them

Normalized Mutual Information: Example



Found communities (H)

- [1,1,1,1,1,1, 2,2,2,2,2,2,2,2]

Actual Labels (L)

– [2,1,1,1,1,1, 2,2,2,2,2,2,1,1]

$n = 14$

	$\mathbf{n}_h$
$h=1$	6
$h=2$	8

	$\mathbf{n}_l$
$l = 1$	7
$l = 2$	7

$\mathbf{n}_{h,l}$	$l = 1$	$l = 2$
$h=1$	5	1
$h=2$	2	6

Evaluation without Ground Truth



(a) U.S . Constitution



(b) Sports

- **Evaluation with Semantics**
 - Analyze **posts, profile information, hashtags, and generated content** of community members.
 - Check whether members of the same community show **semantic/topic coherence**.
 - Extract **frequent keywords** from each community.
 - Human evaluators can judge whether the keywords represent a **meaningful and coherent topic**.
 - Large-scale evaluation can be conducted using **online human-judgment platforms** such as Amazon Mechanical Turk.
 - **Example:**
football, match, goal, FIFA, player → **High topic coherence**
 - **Evaluation Using Clustering Quality Measures**
 - Evaluate communities using **quantitative clustering/community-quality measures**.
 - Run **multiple community detection algorithms** on the same network.
 - Compare their evaluation scores.
 - Select the algorithm with the **better quality according to the chosen measure**.
 - **Examples:**
 - SSE — cluster compactness (*mainly for vector-based clustering*)
 - Modularity — community structure quality
 - Conductance — separation between communities
 - NMI— agreement with ground-truth communities
- Use both **semantic coherence**

Use both **semantic coherence** and **quantitative measures** to obtain a more reliable evaluation of detected communities.

Machine Learning Algorithms for Community Detection

- In the context of **community detection**, certain **machine learning algorithms** can be applied to network data, especially when the problem involves clustering nodes based on features or embeddings derived from the graph.
- These algorithms often extend beyond pure graph-based methods and can utilize data representations, embeddings, or other features to detect communities.
- Below are some machine learning-based algorithms that are commonly used for **clustering and community detection**:

Machine Learning Algorithms for Community Detection

- **1. K-Means Clustering**

- **How it works:** K-Means is a classical unsupervised machine learning algorithm used for clustering. It works by partitioning the data points (nodes) into **k clusters** by minimizing the within-cluster variance. In the context of networks, nodes can be represented as vectors (e.g., node embeddings), and K-Means can be used to cluster these vectors into communities.
- **Strength:** Works well for large datasets with clear separations. Simple and efficient.
- **Example:** Clustering users in a social media network based on features like user activity, profile data, or interactions.
- **Use Case:** Community detection when nodes have feature representations, such as **social networks**, **e-commerce** data, or **biological networks**.

Machine Learning Algorithms for Community Detection

- **2. Gaussian Mixture Models (GMM)**

- **How it works:** GMM is a probabilistic clustering model that assumes data points are generated from a mixture of Gaussian distributions. Each data point is assigned a probability of belonging to each cluster. For community detection, nodes are often represented by feature vectors, and GMM clusters these nodes into communities.
- **Strength:** Provides soft clustering, meaning nodes can belong to multiple clusters with varying probabilities.
- **Example:** Soft clustering of users in a marketing network based on browsing behavior or product preferences.
- **Use Case:** Soft community detection in networks with overlapping features or continuous data distributions.

Machine Learning Algorithms for Community Detection

- **3. DBSCAN (Density-Based Spatial Clustering of Applications with Noise)**
 - **How it works:** DBSCAN clusters data based on density. It groups points that are closely packed together, and marks points that are in low-density regions as noise. For community detection, DBSCAN can be applied to nodes represented by feature vectors.
 - **Strength:** Does not require the number of clusters to be specified in advance and can handle noisy data.
 - **Example:** Detecting communities in a scientific collaboration network, where some researchers (nodes) are outliers or loosely connected.
 - **Use Case:** Networks with non-uniform densities or noisy data (e.g., **anomaly detection in social networks**).

Machine Learning Algorithms for Community Detection

- **4. BERT for Topic Modeling (BERTopic)**

- **How it works:** BERTopic leverages **BERT embeddings** (Bidirectional Encoder Representations from Transformers) and clustering techniques (like UMAP for dimensionality reduction and HDBSCAN for clustering) to identify topics in text data. In the context of community detection, it can be used to find communities based on text-based interactions or communication patterns between nodes (e.g., users, authors, documents).
- **Strength:** Highly effective for clustering text data in networks (e.g., clustering users based on their messages, posts, or reviews).
- **Example:** Detecting communities of users discussing similar topics on social media (e.g., Twitter, Reddit).
- **Use Case:** Community detection in **text-based networks**, such as social media, news articles, or research publications.

Machine Learning Algorithms for Community Detection

- **5. Latent Dirichlet Allocation (LDA) for Topic Modeling**

- **How it works:** LDA is a probabilistic model used for topic extraction in text data. In the context of networks, LDA can be used to cluster nodes based on the topics they are associated with (e.g., clustering authors by the topics of their publications).
- **Strength:** Identifies communities in text-based networks by grouping nodes with similar content.
- **Example:** Clustering researchers in a citation network based on the topics of their published papers.
- **Use Case:** Detecting topic-based communities in **academic networks, social media, or e-commerce reviews**.

Machine Learning Algorithms for Community Detection

- **6. Node2Vec**

- **How it works:** Node2Vec is a **graph embedding algorithm** that transforms nodes into feature vectors based on their graph neighborhood. Once nodes are embedded into a vector space, standard clustering algorithms (like K-Means or GMM) can be applied to detect communities.
- **Strength:** Preserves both local and global network structure in the embeddings.
- **Example:** Detecting communities in a **social network** where users are represented by their connection patterns.
- **Use Case:** Works well in networks where both node features and structural information are important (e.g., social networks, citation networks).

Machine Learning Algorithms for Community Detection

- **7. DeepWalk**

- **How it works:** DeepWalk is a graph embedding technique similar to Node2Vec. It generates node embeddings by performing random walks on the graph, creating sequences that are treated like sentences in natural language processing. These sequences are used to embed nodes into a lower-dimensional space, where clustering algorithms can be applied to detect communities.
- **Strength:** Focuses on capturing the global structure of the graph through random walks.
- **Example:** Detecting communities in a **telecommunication network** based on the communication patterns between users.
- **Use Case:** Effective for large-scale networks with complex global structures, such as **communication** or **transportation networks**.

Machine Learning Algorithms for Community Detection

- **8. Graph Convolutional Networks (GCN)**

- **How it works:** GCNs are a class of neural networks designed for learning representations from graph-structured data. GCNs aggregate information from a node's neighbors and can be trained to predict node labels (communities) based on this information.
- **Strength:** Powerful for semi-supervised or supervised community detection, where labels for some nodes are known.
- **Example:** Detecting communities in a citation network where some research papers are already labeled by field of study.
- **Use Case:** Ideal for **semi-supervised community detection** where node labels are partially available, such as in academic or product networks.

- **9. Autoencoders for Graphs (Variational Graph Autoencoders)**

- **How it works:** Variational autoencoders (VAEs) can be extended to graphs for unsupervised learning. They learn to encode the graph structure into a latent space, from which they can reconstruct the graph. The embeddings learned in the latent space can then be clustered using standard techniques like K-Means.
- **Strength:** Can capture non-linear relationships in the graph structure.
- **Example:** Detecting communities in a social network where interactions are non-linear or complex.
- **Use Case:** Effective for detecting communities in complex, **non-linear networks** such as **financial networks** or **biological networks**.

Machine Learning Algorithms for Community Detection

- **10. Hierarchical Dirichlet Process (HDP)**

- **How it works:** HDP is a non-parametric Bayesian method used for hierarchical topic modeling. In community detection, it can be used to discover hierarchical community structures by clustering nodes at multiple levels of abstraction.
- **Strength:** Finds hierarchical communities without needing to specify the number of communities in advance.
- **Example:** Detecting hierarchical communities in a research network, where papers are grouped into general fields and subfields.
- **Use Case:** Hierarchical community detection in **academic networks, e-commerce, or biological systems.**

Summary

Algorithm	Description	Strength	Use Case
K-Means	Partitions nodes into k clusters by minimizing within-cluster variance	Simple, efficient for large datasets	Social media networks, user attributes clustering
Gaussian Mixture Model (GMM)	Probabilistic clustering, nodes belong to clusters with certain probabilities	Soft clustering, handles overlapping communities	Marketing, scientific networks with probabilistic connections
DBSCAN	Density-based clustering, detects clusters of varying densities	Handles noise and non-uniform densities	Social networks, anomaly detection
BERTopic	Clusters text data using BERT embeddings and topic modeling	Powerful for text-based community detection	Text-based networks, social media, research papers
LDA	Topic modeling for clustering based on text	Finds topics and communities based on content	Academic networks, social media
Node2Vec	Learns feature representations of nodes using graph-based random walks	Preserves local and global structure, effective for large networks	Social networks, citation networks
DeepWalk	Embeds nodes based on random walks to capture global structure	Focuses on capturing global structure	Communication networks, transportation networks
Graph Convolutional Networks	Learns node embeddings by aggregating neighborhood information	Ideal for semi-supervised tasks	Citation networks, semi-supervised community detection
Autoencoders (VGAE)	Learns non-linear embeddings from graph structure	Captures non-linear relationships in the graph	Complex networks (financial, biological)
Hierarchical Dirichlet Process	Finds hierarchical communities through Bayesian topic modeling	Automatically detects hierarchical community structures	Hierarchical networks like academic, e-commerce